

EMG Signal Pre-Processing and Analysis

Assignment Report

Name Rith Rajak

Roll No. 24116085

April 19, 2026

1 Introduction

EMG (Electromyography) records the electrical activity produced by muscles when they contract. When you put surface electrodes on skin, you are picking up a combined signal from many motor units firing at the same time. The problem is that the electrodes also pick up a lot of unwanted stuff – low frequency drift from movement, powerline noise, and high frequency electronic noise from the amplifier. So before you can do any useful analysis, the signal needs to be cleaned up.

This notebook covers that whole pipeline: first bandpass filtering to keep only the frequencies where the real EMG signal lives, then rectification and envelope detection to get a smooth representation of muscle activity level, and finally threshold-based detection to find when contractions start and stop. This report goes through each stage and answers the study questions.

2 Band-Pass Filtering

2.1 What the code does

```
1 b, a = butter(2, ([10, 400]/(xx['freq']/2)), btype='bandpass')  
2 dataf = filtfilt(b, a, xx['waveData'])
```

A 2nd order Butterworth bandpass filter is applied between 10 Hz and 400 Hz using `filtfilt`. The cutoffs are normalised by dividing by the Nyquist frequency (`xx['freq']/2`), which is just how `scipy` expects the input – you give it a value between 0 and 1 where 1 is Nyquist.

The choice of `filtfilt` over `lfilter` is important. `lfilter` applies the filter in one direction and introduces a phase shift – different frequencies get delayed by different amounts, which distorts the shape of the signal. `filtfilt` does the filter pass twice (forward and backward) so the phase shifts cancel out. For EMG this matters because if the shape or timing of the signal is distorted you get errors in the onset detection later.

Q1: What are the effects of bandpass filtering?

After filtering, the signal looks noticeably cleaner. The slow baseline drift (from movement or electrode sway) disappears because those are below 10 Hz and get cut. The high frequency noise floor also drops since everything above 400 Hz is removed. During rest periods the signal is much flatter, and the contraction bursts stand out more clearly against the baseline. The amplitude during contractions stays roughly the same because the main EMG energy is inside the 10–400 Hz range.

Q2: Why 10–400 Hz? How does this relate to the physiology?

The frequency range has to match where the actual muscle signal lives. EMG signals are made up of Motor Unit Action Potentials (MUAPs) – brief voltage spikes from individual muscle fibres. These spikes have a duration of roughly 3–15 ms, which means their frequency content spans from about 10 Hz up to around 400–500 Hz, with most energy concentrated around 50–150 Hz.

So 10 Hz is chosen as the lower limit to keep the slow onset and offset parts of MUAPs while rejecting motion artefact (which is mostly below 5 Hz) and any ECG contamination from heart activity (which is in the 1–5 Hz range). 400 Hz is chosen as the upper limit to capture the full MUAP spectrum while cutting out high frequency noise. If the sampling rate were different you would need to adjust these, which is why the code normalises against the actual sampling rate rather than hardcoding a number.

Q3: What happens if you change the frequency band?

If you narrow the band, say 50–200 Hz, you cut off some real MUAP content at both ends. The signal looks smoother, but you are actually losing information – weaker contractions might get attenuated to the point where they are hard to detect. The lower cutoff is probably the more sensitive one: dropping it below 10 Hz starts letting in motion artefact and makes the baseline wander again. Raising it above 20 Hz starts cutting into low-frequency MUAP tails and reduces the apparent amplitude of contractions.

On the other side, if you widen the band to something like 1–1000 Hz, you let in more artefact. Below 5 Hz you start seeing slow baseline drift. Above 500 Hz (depending on the sampling rate) you are mostly just adding back electronic noise. There is no advantage to going beyond the MUAP frequency range, you just add noise to the signal.

Q4: What about low SNR recordings?

The bandpass filter will still clean up everything outside the 10–400 Hz range, but it cannot help with noise that falls inside the band. The main problem with a low SNR recording is usually in-band noise – things like 50 Hz or 60 Hz powerline interference, which sits right inside the passband. For those cases you would need to add a notch filter specifically at 50/60 Hz on top of the bandpass. The filtered signal would then still have a higher noise floor than a clean recording because whatever in-band noise remains cannot be removed without also removing real signal, so onset detection becomes harder to do reliably.

Q5: Other filter types

A **notch filter** removes a very narrow frequency band, typically 50 or 60 Hz for powerline interference. It is usually used in addition to a bandpass, not instead of it – it handles one specific noise source that the bandpass cannot remove because that noise falls inside the passband.

A **Chebyshev filter** would give a steeper rolloff than Butterworth (it cuts more sharply at the edge frequencies), but introduces ripple in the passband, meaning it slightly amplifies or attenuates different frequencies within the kept range. For EMG analysis where amplitude matters, this is a problem.

Wavelet-based filtering decomposes the signal into different frequency scales and lets you selectively zero out or shrink coefficients at noisy scales, similar to what was done in the ECG notebook. It handles non-stationary noise better than a fixed Butterworth filter – if the noise changes character over time, the wavelet approach adapts. The tradeoff is it is more complex to implement and has more parameters to tune.

3 Rectification and Envelope Detection

3.1 Rectification

```
1 absSignal = np.absolute(dataf)
```

The filtered signal oscillates around zero – it has both positive and negative deflections. If you tried to compute an envelope directly by low-pass filtering the raw signal, the positive and negative halves would partially cancel out and you would get something close to zero even during a strong contraction. Rectification fixes this by taking the absolute value so all the signal energy is on the positive side. Then the low-pass filter can meaningfully average it into an envelope.

3.2 Envelope detection

```
1 lowp = 8
2 low_pass = lowp/sfreq
3 b, a = sc.signal.butter(4, low_pass, btype='lowpass')
4 datafrle = filtfilt(b, a, absSignal)
```

A 4th order Butterworth low-pass filter at 8 Hz is applied to the rectified signal.

Q1: Effects of low-pass filtering on the rectified signal

The low-pass filter smooths out all the rapid spikes in the rectified signal and leaves a slow, smooth curve that tracks the overall level of muscle activity. Before filtering, the rectified signal is a rapidly oscillating signal showing individual MUAPs. After, you get a clean hump for each contraction that rises gradually as the muscle activates and falls back to near zero during rest. The fine structure of individual motor units is gone – what you see is just the amplitude envelope.

Q2: Why 8 Hz?

Voluntary muscle contractions are slow events compared to the EMG signal itself. When a person intentionally contracts and relaxes a muscle, the force level changes at a rate of a few Hz at most – the neuromuscular system cannot modulate force faster than around 3–8 Hz under normal voluntary control. So a cutoff of 8 Hz is just high enough to track all the real changes in muscle activity while removing the rapidly oscillating MUAP structure that is not useful for detecting when contractions happen.

If the cutoff were too low, say 2 Hz, the envelope would respond slowly and the estimated start and stop times would be off – the envelope would not rise fast enough at onset or fall fast enough at offset. If it were too high, say 30 Hz, the envelope would still be noisy and have oscillations that could cross the threshold multiple times during a single contraction, giving false start/stop events.

Q3: What happens if you change the cutoff?

Lowering it to around 2–3 Hz: the envelope becomes very sluggish. The rise at contraction onset is delayed and the fall after offset is stretched out. For a task with slow sustained contractions this is somewhat okay, but for detecting the exact timing of onset it introduces systematic error – the detected start is always a bit late and the stop is always a bit early. Duration estimates get shorter than they should be.

Raising it to 30–50 Hz: the envelope starts looking more like a smoothed rectified signal with visible oscillations on top of the contraction humps. The threshold will be crossed multiple times per contraction, so the code would detect several spurious start/stop pairs within a single real contraction. This makes the whole analysis unreliable.

Q4: Other envelope techniques

RMS (Root Mean Square) with a sliding window is probably the most common alternative. You compute the square root of the mean squared signal in a short sliding window:

```
1 window_ms = 125 # ms window
2 window_samples = int(window_ms / 1000 * sfreq)
3 rms_env = np.array([
4     np.sqrt(np.mean(dataf[i : i + window_samples]**2))
5     for i in range(len(dataf) - window_samples)
6 ])
```

RMS has a physical meaning – it is proportional to signal power – and is more directly related to muscle force than the low-pass filtered absolute value. The window length plays the same role as the low-pass cutoff: a 125 ms window at 8000 Hz gives roughly 8 Hz effective bandwidth. Most published EMG studies use RMS rather than the low-pass approach.

The disadvantage is that a simple loop like above is slow for long signals. You would normally use a convolution or cumulative sum trick to compute it efficiently.

Hilbert transform gives an instantaneous amplitude without needing rectification first. `scipy.signal.hilbert` returns the analytic signal and you take its magnitude. It is mathematically clean and has no window or cutoff parameter to tune. The main problem is it is sensitive to edge effects at the start and end of the recording. Also, it is not commonly used in EMG analysis so results are less comparable to published literature.

4 Contraction Detection

4.1 How the threshold detection works

```
1 threshold = 200
2 cstarts = []
3 for i in range(1, len(datafrle)):
4     if datafrle[i-1] < threshold and datafrle[i] >= threshold:
```

```
5         cstarts.append(i)
6
7     cstops = []
8     for i in range(1, len(datafrle)):
9         if datafrle[i-1] >= threshold and datafrle[i] < threshold:
10            cstops.append(i)
```

This is a simple edge detector. For starts, it looks for samples where the envelope was below the threshold in the previous step and is now at or above it – the first upward crossing. For stops, the reverse – the first downward crossing. The threshold of 200 is set by looking at the envelope plot and picking a value that sits above the resting noise but below the contraction peaks.

Q1: What happens when you change the threshold?

Decreasing the threshold: the envelope crosses it sooner on the way up and later on the way down, so detected durations get longer. More importantly, small post-contraction twitches and baseline noise fluctuations that were previously below the threshold now cross it and get detected as contractions. The start/stop lists get longer with extra spurious events, and some of these pairs will have very short durations which are obviously noise.

Increasing the threshold: the opposite – durations get shorter, onset is detected later, offset is detected earlier. Weak contractions might never reach the threshold and get missed entirely. In a fatigue experiment where contraction amplitude drops over time, using too high a threshold would mean later contractions in the recording are systematically missed.

Q2: Disadvantages of this detection method

There are a few real problems with this approach.

First, the threshold is a fixed absolute value in raw signal units, which means it only makes sense for the specific recording it was set on. Different subjects, different electrode placements, different skin impedance – all of these change the absolute amplitude of the EMG signal. A threshold of 200 that works for one recording could miss all contractions in another or generate all false detections in a third. There is no automatic way to determine a good threshold without looking at the signal.

Second, there is no minimum duration check. A brief noise spike that crosses the threshold for just one or two samples creates a start-stop pair with a near-zero duration. These end up in the duration list and mess up any statistics you compute.

Third, the code builds two independent lists – starts and stops – and assumes they are the same length. If the recording starts mid-contraction (so a stop event appears before any start), or ends mid-contraction (a start with no matching stop), the lists are unequal. The duration loop `stops[n]-starts[n]` would then pair the wrong events silently, giving nonsense durations without any error.

Fourth, if the envelope dips briefly below the threshold in the middle of a sustained contraction – even just one sample – the code treats it as the end of one contraction and the start of another. A single wobble in the envelope splits what is really one continuous contraction into two.

Q3: Better methods for onset detection

Double threshold / hysteresis: Use a higher threshold to detect onset and a lower one to detect offset. The contraction is considered to have started when the envelope crosses the high threshold, and ended only when it falls below the lower one. This means brief dips in the envelope during a contraction do not trigger a false offset, which is the main problem with the single threshold approach.

Statistics-based threshold: Instead of manually picking 200, estimate the mean and standard deviation of the envelope during a known rest period at the start of the recording, then set the threshold at $\text{mean} + 3 \cdot \text{std}$ (or some multiple). This makes the threshold adaptive to each recording's noise level automatically and removes the need for manual inspection.

TKEO (Teager-Kaiser Energy Operator): This is a nonlinear operator that computes $\Psi[x(n)] = x(n)^2 - x(n-1) \cdot x(n+1)$ and is very sensitive to sudden changes in amplitude. It responds faster than a low-pass envelope and can detect onset to within a few milliseconds, which matters if you are measuring reaction times or the timing of muscle coordination. The tradeoff is it is noisier and more sensitive to any remaining high-frequency content in the signal.

Q4: Quantifying rest intervals

The rest interval is just the time from the end of one contraction to the start of the next. With the existing `starts` and `stops` lists this is straightforward:

```
1 rest_intervals = []
2 for n in range(0, len(starts) - 1):
3     rest = starts[n+1] - stops[n]
4     rest_intervals.append(round(rest, 2))
5     print(f'Rest interval {n+1}: {round(rest, 2)} s')
6
```

```
7 print(f'Mean rest: {round(np.mean(rest_intervals), 2)} s')  
8 print(f'Std rest: {round(np.std(rest_intervals), 2)} s')
```

For each contraction (except the last), the rest interval is `starts[n+1] - stops[n]`. Printing the mean and standard deviation is useful for checking how consistently the subject followed the protocol. If the rest intervals shorten over time it could indicate that the subject was not recovering fully between contractions, which would affect any fatigue analysis. A large standard deviation in rest times also makes it harder to compare contraction durations fairly, since contractions following shorter rests might be weaker due to incomplete recovery.

5 Overall Assessment

The pipeline works well as a starting point for EMG analysis. The filter choices are sensible – 10–400 Hz is a standard bandpass range for surface EMG, and using `filtfilt` is the right call to avoid phase distortion. The rectify-then-lowpass approach for envelope detection is simple and effective, even if RMS would be the more rigorous choice.

The main weakness is that everything depends on manually chosen parameters that only work for the specific recording being analysed. The threshold of 200, the choice of 8 Hz for the envelope, and even the 10–400 Hz bandpass would all need to be revisited for a different subject or a different electrode placement. For a single demonstration this is fine, but for processing a larger dataset automatically you would need adaptive or normalised versions of each step.

The duration calculation also has a silent bug: if `len(starts) != len(stops)` for any reason, the pairs are misaligned and the durations are wrong. Adding a check for this before computing durations would make the code more reliable.